

GSFlow: Video Rendering Acceleration for 3D Gaussian Splatting via Geometry Pixel Flow

Yehao Lu, Qi Zhao, Ziyu Lv, Rui Jiang, Xuewei Li, Zequn Qin*, Xi Li*

Abstract—Recent advances in accelerating 3D Gaussian Splatting (3DGS) rendering, such as pruning, rasterization optimization, and attribute compression, have primarily focused on frame-level acceleration to minimize single-view latency. However, these methods overlook the strong temporal coherence inherent in continuous camera trajectories, leading to redundant computations as highly similar adjacent frames are rendered independently from scratch. Inspired by this insight, we propose GSFlow, a novel sequence-level acceleration paradigm orthogonal to existing frame-level methods. Specifically, GSFlow samples key frames at fixed intervals along the camera trajectory and renders their RGB and depth images via 3DGS. For subsequent frames, it lifts keyframe RGB pixels to world coordinates and projects them onto target frames, achieving geometry pixel flow. To address blank pixels from camera motion and projection errors, we propose two solutions: a lightweight inpainting network for higher rendering quality, and a training-free filtering-based method accelerated via customized CUDA parallelization for extreme speed. For edge flickering caused by depth inaccuracies, we propose a self-supervised depth calibration strategy that enhances spatio-temporal consistency by inter-frame 3D geometry alignment. Extensive experiments demonstrate that GSFlow achieves an additional $\times 2$ speedup over state-of-the-art 3DGS acceleration methods while maintaining rendering quality. Code will be made publicly available.

I. INTRODUCTION

3D Gaussian Splatting (3DGS) [1], [2], [3], [4], [5] has demonstrated remarkable success in 3D scene reconstruction and novel view synthesis by achieving photorealistic results with impressive efficiency. It has facilitated a broad spectrum of applications, including autonomous driving [6], [7], [8], [9], augmented and virtual reality (AR / VR) [10], [11], [12], [13], and robotics [14], [15], [16], [17]. These real-time applications demand highly efficient rendering to ensure smooth human-computer interaction on limited computational devices while maintaining low power consumption to prevent overheating. However, accelerating 3DGS rendering under such constrained resources remains a critical challenge.

Video rendering for 3DGS involves rendering a continuous camera trajectory. Current acceleration methods focus on the frame level, aiming to minimize rendering latency for individual view but ignore the strong temporal coherence between consecutive frames. It results in substantial computational waste, as highly similar adjacent frames are rendered

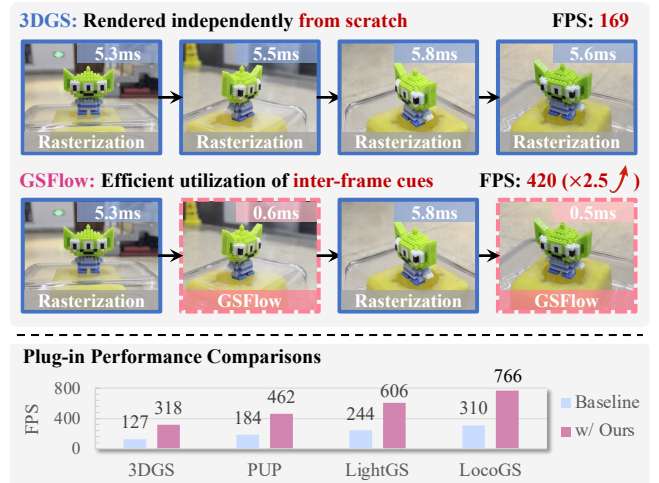


Fig. 1. GSFlow is an efficient sequence-level acceleration framework for 3D Gaussian Splatting (3DGS) rendering. It leverages inter-frame coherence to minimize the number of pixels requiring 3DGS rendering, thereby achieving significant speedup. Our GSFlow can be training-free and directly applied to 3DGS checkpoints with their inherent depth and is orthogonal to existing frame-level methods. When integrated with state-of-the-art baselines, GSFlow achieves an average $\times 2$ FPS speedup at a 2-frame projection interval.

completely from scratch without reusing inter-frame information. Representative methods include Compact-3DGS [18] and LightGaussian [19] reduce the computational load by pruning 3D Gaussian primitives. FlashGS [20] and Speedy-Splat [21] further accelerates rendering via optimizing the rasterization process. ScaffoldGS [22] and HAC [23] enhance efficiency by exploring the structured representations of 3D Gaussians. Despite their varied techniques, all of these methods focus on frame-level acceleration. Based on this analysis, we shift the research focus to the sequence level. Our key insight is to leverage inter-frame coherence to reduce the number of pixels requiring 3DGS rendering, thereby achieving significant speedup. It is worth noting that this establishes an acceleration paradigm orthogonal to existing frame-level methods, enabling multiplicative performance improvement.

In this work, we propose GSFlow, a novel sequence-level rendering acceleration framework for 3DGS. Specifically, we propose a geometry pixel flow as shown in fig. 2. We first sample key frames at fixed intervals and use the pre-trained 3DGS model to render the corresponding RGB and depth images. By leveraging the camera parameters of the key frames, we lift RGB pixels to 3D world coordinates. These lifted 3D color points are then projected into subsequent frames using their camera parameters, achieving pixel motion propagation through the sequence. However, this process

Yehao Lu, Qi Zhao, Ziyu Lv, Rui Jiang and Xi Li are with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310007, China. Zequn Qin is with the School of Software Technology, Zhejiang University, Ningbo 315048, China. Xuewei Li is with the School of Electronic and Information, Shanghai Dianji University, Shanghai 201306, China. E-mail: {luyehao, qizhao, 3220104968, jrss, qinzequn, xilizju}@zju.edu.cn, xueweili@sdju.edu.cn.

(Corresponding author: Zequn Qin, Xi Li)

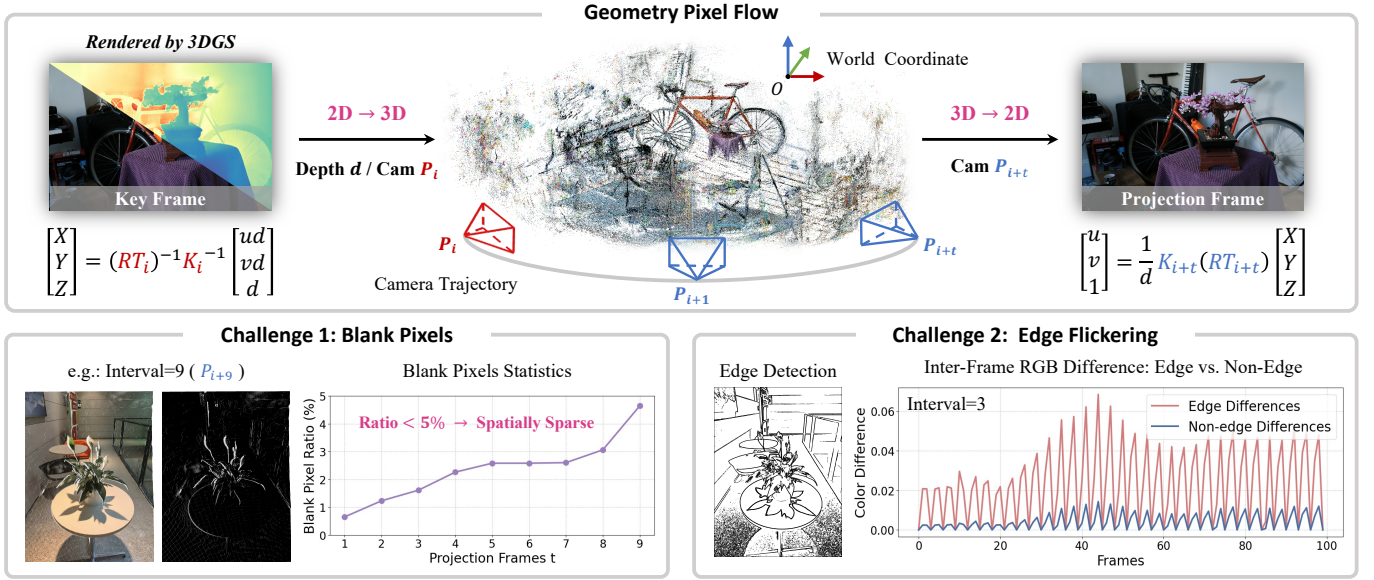


Fig. 2. **Illustration of GS-Flow.** The proposed GSFlow samples key frames at fixed intervals and renders their RGB and depth maps via 3DGS model. For subsequent frames, it lifts keyframe RGB pixels to 3D world coordinates and projects them onto target frames, achieving geometry pixel flow. However, it faces two key challenges. **First, Blank Pixels.** Certain areas in the projection frames remain unprojected due to depth errors. Fortunately, these pixels are sparse. Even at a 9-frame interval in 30 FPS video, the projection ratio remains above 95%. **Second, Edge Flickering.** Larger color-brightness discrepancies are observed in edge regions when compared to 3DGS-rendered images, occasionally resulting in flickering during continuous camera trajectory rendering. This work effectively addresses these challenges, achieving sequence-level rendering acceleration while maintaining high-fidelity visual quality.

inherently involves two critical challenges.

First challenge is blank pixels. Camera motion and projection errors cause blank pixels of subsequent frames, as shown in fig. 2 (bottom-left). Nevertheless, such pixels are sparse. For instance, even at a 9-frame projection interval in 30 FPS video, the blank pixel ratio remains below 5%. To efficiently inpaint these pixels, we propose two solutions. The first utilizes a lightweight inpainting network conditioned on the projected image, key frame, and projection mask. This conditioning strategy, inspired by diffusion models [24], efficiently reduces learning difficulty while enhancing fidelity and generalization. The second employs a training-free filtering-based approach accelerated by a custom CUDA implementation, where an efficient search algorithm is designed to locate the nearest projected pixels for inpainting, achieving extreme speed.

Second challenge is edge flickering. Due to the depth discontinuity at object edges, a portion of RGB pixels are projected to incorrect locations, causing inter-frame inconsistency and noticeable flickering artifacts. As shown in fig. 2 (bottom-right), object edges exhibit a significantly larger color discrepancy relative to the ground truth than non-edge regions. To mitigate this, we propose a self-supervised depth calibration strategy that enhances spatio-temporal consistency by minimizing the Euclidean distance between the corresponding projected RGB points between frames in world coordinates during 3DGS training. It significantly stabilizes the motion propagation.

Extensive experiments demonstrate that our GSFlow achieves a $\times 2$ FPS speedup over various existing accelerated 3DGS methods while maintaining comparable rendering quality and high inter-frame consistency. Besides, the computational cost of our method is highly predictable and hardware-friendly, as it scales linearly by the key frame sampling

interval, which indicates the number of frames skipping 3DGS rasterization.

Our contributions can be summarized as:

- We propose a novel sequence-level acceleration framework for 3DGS rendering that is orthogonal to existing frame-level methods, achieving an average $\times 2$ speedup when integrated with state-of-the-art baselines.
- We propose a geometry pixel flow that leverages inter-frame similarity to efficiently synthesize adjacent frames, achieving an $\times 8.83$ speedup over 3DGS rasterization while significantly reducing computational load and power consumption with highly predictable cost.
- We propose two efficient blank pixel inpainting methods: a generative network for higher rendering quality, and a training-free filter accelerated by our customized CUDA for extreme speed, along with a self-supervised calibration strategy to enhance spatio-temporal consistency.

II. RELATED WORK

A. 3D Gaussian Pruning

Recent research has revealed that a large portion of Gaussian primitives are redundant [19], [25], [26], motivating a series of methods that boost rendering speed by reducing the number of Gaussians with minimal loss of visual fidelity. CompGS [27] and Compact-3DGS [18] prioritizes pruning low-opacity Gaussians. EAGLES [28] prunes Gaussians based on the least total transmittance. LightGaussian [19] further incorporates Gaussian volumes into the importance computation. Besides, Mini-Splatting [25] and RadSplat [26] introduce heuristics-based pruning strategies. PUP 3D-GS [29] uses Fisher Information matrix to calculate the importance scores, achieving high compression ratios. APNet [30] employs adversarial

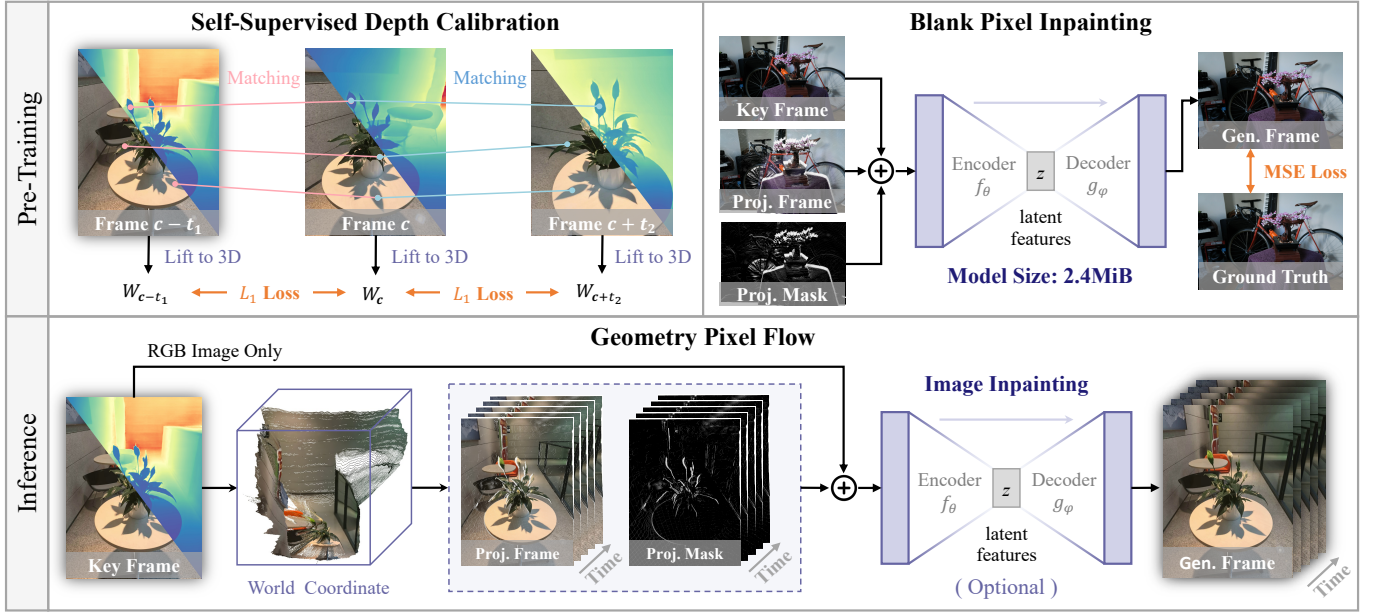


Fig. 3. **Overview of GS-Flow.** The pre-training stage comprises training a 3DGS model $\mathcal{G}$ and a generative inpainting network $\mathcal{F}$. For training $\mathcal{G}$, we propose a self-supervised depth calibration strategy that enhances spatio-temporal consistency by 3D geometry alignment. For training $\mathcal{F}$, we inject not only the projected image but also the key frame and projection mask as additional conditions, which reduces the learning difficulty and improves the inpainting quality and generalization capability. During inference, we utilize the geometry pixel flow mechanism and the alternative inpainting method to quickly generate images for the subsequent views of key frames.

learning to balance the reduction of redundant Gaussians with the preservation of visual fidelity.

B. Rasterization Optimization

In addition to pruning Gaussian primitives, another line of research focuses on optimizing the rasterization. AdR-Gaussian [31], StopThePop [32], FlashGS [20] and Speedy-Splat [21] introduce precise tile intersection methods to reduce the number of Gaussians processed per tile. Besides, Balanced-3DGS [33] proposes a Gaussian-wise parallelism rendering with fine-grained tiling, solving workload-imbalance among pixels and Gaussian spheres. SEELE [12] proposes a contribution-aware rasterization, which dynamically assigns computational budgets to Gaussians, thereby improving the GPU utilization.

C. Attribute Compression

Several works have employed compression techniques such as vector quantization (VQ) [18], [19], [27], [28], [34], [35] and entropy coding [18], [23], [36], [37], [38], [39] to improve efficiency and reduce storage. In parallel, another line of work has explored structured representations. ScaffoldGS [22] introduces anchor points to distribute local 3D Gaussians, and predicts their attributes based on the anchor features and the viewing positions. HAC [23] and HAC++ [40] further integrates anchors with a structured hash grid to improve compression rates. ContextGS [41] pioneers the context model at the anchor level for 3DGS compression, achieving an impressive size reduction. Besides, OMG [35] combines pruning, quantization, and attribute representation to achieve ultra-high Gaussian compression rates while maintaining high-fidelity rendering.

Despite the varied techniques of these approaches, they primarily focus on frame-level acceleration for 3DGS, aiming to optimize single-view latency but ignoring temporal consistency. In contrast, this paper explores a sequence-level acceleration paradigm, which is orthogonal to frame-level methods and achieves further acceleration.

III. METHODS

A. Preliminaries

In this work, we aim to accelerate the 3DGS rendering of a continuous camera trajectory $P = \{P_1, P_2, \dots, P_n\}$, where n denotes the number of views. 3DGS models the 3D scene as a set of anisotropic 3D Gaussians $\mathcal{G}$. Each Gaussian is represented as $\mathcal{G}_i = \{\mu_i, r_i, s_i, c_i, o_i\}$, including mean position $\mu_i \in \mathbb{R}^3$, rotation quaternion $r_i \in \mathbb{R}^4$, anisotropic scaling factors $s_i \in \mathbb{R}^3$, color $c_i \in \mathbb{R}^3$ computed from spherical harmonics (SH) with the view direction, and opacity constant $o_i \in (0, 1)$. The covariance matrix of each Gaussian $\Sigma_i \in \mathbb{R}^{3 \times 3}$ is determined by s_i and r_i . To render an image from a given viewpoint, the color $C(x)$ of each pixel is calculated by α -blending:

$$C(x) = \sum_{i=1}^{\mathcal{N}(x)} c_i \alpha_i(x) \prod_{j=1}^{i-1} (1 - \alpha_j(x)) \quad (1)$$

$$\alpha_i(x) = o_i \exp \left(-\frac{1}{2} (x - \mu'_i)^T \Sigma_i'^{-1} (x - \mu'_i) \right) \quad (2)$$

where x denotes a pixel coordinate, μ'_i and Σ_i' represent the projected 2D center position and Gaussian covariance matrix of $\mathcal{G}_i$. $\mathcal{N}(x)$ is the number of Gaussians intersecting with pixel x , where these Gaussians are depth-sorted based on the given view direction.

B. Overview

In this work, we propose a geometry pixel flow mechanism that rapidly synthesizes adjacent frames via inter-frame projection. The overall pipeline is depicted in fig. 3. This approach faces two key challenges including blank pixels and edge flickering. To address blank pixels, we propose two alternative solutions. The first utilizes a lightweight inpainting network $\mathcal{F}$. For training $\mathcal{F}$, we first construct a blank-pixel completion dataset. Then, by feeding key frames $I_k \in \mathbb{R}^{H \times W \times 3}$, projected frames $I_p \in \mathbb{R}^{H \times W \times 3}$, and projection masks $M_p \in \mathbb{R}^{H \times W}$ as conditions into $\mathcal{F}$, we lower the learning barrier and markedly enhance its inpainting performance and generalization ability. The second employs a training-free filtering-based inpainting method. It uses an efficient search algorithm to locate the nearest projected pixels for inpainting, accelerated by a customized CUDA implementation. To mitigate edge flickering, we introduce a self-supervised calibration strategy during training 3DGS model $\mathcal{G}$ to enhance spatio-temporal consistency. For training $\mathcal{G}$, we align the 3D spatial coordinates of corresponding pixels between the current frame and adjacent frames via a calibration loss, effectively reducing temporal flickering caused by depth errors and significantly improving rendering stability. During inference, we utilize the 3DGS model to render RGB images I_k and depth maps $D_k \in \mathbb{R}^{H \times W}$ of key frames, lift these pixels to the world coordinate system, and project them onto subsequent frames I_p . The blank regions are efficiently inpainted using one of the proposed inpainting methods. This approach directly replaces the complex sorting and α -blending computations in 3DGS rasterization, achieving significant acceleration with reduced computational overhead.

C. Self-Supervised Depth Calibration

During the training of the 3DGS model $\mathcal{G}$, we select the preceding frame P_{c-t_1} and subsequent frame P_{c+t_2} relative to the current view P_c as neighboring frames, where $t_1, t_2 \in (0, 5]$ are uniformly random sampled integers. Utilizing the rendered RGB images I_c and depth maps D_c of P_c from 3DGS, along with camera intrinsics and extrinsics, we first lift the pixels to the 3D world coordinate system and then project them onto the neighboring frames to establish pixel-level correspondences, which are calculated as follows:

$$W_c = \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = (RT_c)^{-1} K_c^{-1} \begin{bmatrix} u_c d_c \\ v_c d_c \\ d_c \end{bmatrix} \quad (3)$$

$$I_{c-t_1} = \begin{bmatrix} u_{c-t_1} \\ v_{c-t_1} \\ 1 \end{bmatrix} = \frac{1}{d_{c-t_1}} K_{c-t_1} RT_{c-t_1} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} \quad (4)$$

where $K \in \mathbb{R}^{3 \times 3}$ is the camera intrinsic, $RT \in \mathbb{R}^{3 \times 4}$ is the camera extrinsic, $d_c \in D_c$ denotes the depth of the pixel, W_c represents the world coordinates of the lifted pixels in I_c , and I_{c-t_1} is the the projected image onto the neighboring frame P_{c-t_1} . I_{c+t_2} is computed in the same manner. Then, we use 3DGS model $\mathcal{G}$ to render the depth maps of P_{c-t_1} and P_{c+t_2} , respectively. The pixels in I_{c-t_1} and I_{c+t_2} that correspond to those in I_c are lifted to the world coordinate system following

Algorithm 1: Inference Process of GSFlow

Input: 3DGS model $\mathcal{G}$, Network $\mathcal{F}$, Filter $\mathcal{K}$

Output: Rendering video V

```

1 ▷ Key Frame Initialization
2  $P_k \leftarrow P_1, (I_k, D_k) \leftarrow \mathcal{G}(P_k);$ 
3 for  $i \in [2, \text{len}(V)]$  do
4   Obtain camera pose  $P_c$ ;
5    $\vartheta_c \leftarrow \text{ComputeKeyFrame}(P_c)$  via eq. (7);
6   if  $\vartheta_c = 1$  then
7     ▷ Update Key Frame via 3DGS
8      $P_k \leftarrow P_c, (I_k, D_k) \leftarrow \mathcal{G}(P_k);$ 
9     Append  $I_k$  to  $V$ ;
10  else
11    ▷ Geometry Pixel Flow
12     $W_k \leftarrow \text{2Dto3DProj}(P_k, I_k, D_k)$  via eq. (3);
13     $I_c, M_c \leftarrow \text{3Dto2DProj}(P_c, W_k)$  via eq. (4);
14    ▷ Blank Pixel Inpainting (Optional)
15     $I'_c \leftarrow \mathcal{F}(I_c \oplus M_c \oplus I_k)$  or  $I'_c \leftarrow \mathcal{K}(I_c);$ 
16    Append  $I'_c$  to  $V$ ;
17  end
18 end
19 return  $V$ ;
```

eq. (3), obtaining W_{c-t_1} and W_{c+t_2} . The self-supervised depth calibration loss $\mathcal{L}_{\text{calib}}$ is calculated as follows:

$$\mathcal{L}_{\text{calib}} = \frac{1}{M} \left(\|W_c - W_{c-t_1}\|_2 + \|W_c - W_{c+t_2}\|_2 \right) \quad (5)$$

where M denotes the number of valid 3D points, which is slightly less than $H \times W$ due to some points being projected outside the image boundaries. To enhance temporal consistency while preserving rendering quality, the training process also requires to incorporate the image reconstruction loss $\mathcal{L}_{\text{color}}$ and $\mathcal{L}_{\text{D-SSIM}}$ following 3DGS [1].

D. Blank Pixel Inpainting

The projection between frames often yields blank pixels due to camera motion and depth errors, necessitating an image-to-image (I2I) inpainting method. This method must meet two key criteria: (1) High inference speed. Its runtime must be significantly faster than 3DGS rasterization to achieve acceleration. (2) Strong generalization. As 3DGS scenes are highly diverse, this method must generalize across numerous scenes and maintain high rendering quality, avoiding impractical per-scene optimization.

To this end, we propose two alternative solutions. The first employs a lightweight generative inpainting network based on a UNet architecture, which supports arbitrary-resolution inputs and has a compact size of only 2.4MiB. To enhance generalization, we first construct a paired image dataset for blank-pixel completion, covering various scenes and different blank-pixel ratios. Besides, we inject not only the projected image I_p but also the key frame I_k and projection mask M_p as additional conditions into the network. This reformulates the inpainting task as a corresponding pixel search problem on the key frame, reducing learning difficulty while improving

TABLE I

QUANTITATIVE RESULTS OF GSFlow EVALUATED ON THE TANKS&TEMPLES AND OUR CUSTOM VIDEO DATASETS. ALL BASELINE RESULTS ARE REPRODUCED USING THEIR OFFICIAL OPEN-SOURCE CODE AND CONFIGURATIONS. THE FILTERING-BASED INPAINTING METHOD IS UTILIZED. BASELINE METHOD IS LOCOGS [42]. FRAME PROJECTION INTERVAL $T = 3$. FPS IS TESTED ON A SINGLE RTX 4090 GPU.

Methods	Tanks&Temples				Custom Video Dataset			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$
3DGS [1]	31.44	0.926	0.143	92	33.77	0.946	0.160	169
Reduced-3DGS [43]	30.85	0.924	0.413	125	33.44	0.943	0.165	295
LightGaussian [19]	27.24	0.867	0.233	132	32.59	0.934	0.194	193
EAGLES [28]	30.29	0.913	0.172	152	33.44	0.945	0.165	245
PUP 3D-GS [29]	30.88	0.923	0.150	152	33.24	0.943	0.163	266
Taming-3DGS [44]	30.18	0.904	0.186	170	33.67	0.936	0.164	210
Compact-3DGS [18]	29.92	0.911	0.172	168	32.67	0.939	0.175	331
CompGS [27]	29.07	0.884	0.230	197	31.72	0.934	0.192	333
LocoGS [42]	31.23	0.921	0.157	165	32.62	0.933	0.191	421
GSFlow GSFlow (Ours)	30.22	0.915	0.167	310	31.87	0.919	0.211	637

completion quality and generalization. For simplicity, we concatenate these images as input, obtaining the completed image I'_p :

$$I'_p = \mathcal{F}(I_p \oplus M_p \oplus I_k) \quad (6)$$

where $(I_p \oplus M_p \oplus I_k) \in \mathbb{R}^{H \times W \times 7}$. To optimize the network parameters, we adopt a reconstruction MSE loss $\mathcal{L}_{recons}$. Another method uses a training-free, filtering-based completion approach. For each blank pixel, we locate its nearest RGB-projected pixel by progressively expanding a search window, then assign the corresponding color value. Aided by customized CUDA parallel acceleration, this method completes inpainting in just 0.1ms on average.

E. Geometry Pixel Flow

During inference, our method dynamically selects key frames online. The first frame is initially set as a key frame. For each subsequent frame, projection is performed only if it meets the following criteria relative to the current key frame simultaneously: the frame interval difference $\xi_{i,k}$ is less than constant T , the camera distance $\omega_{i,k}$ is less than threshold φ_d , and the viewing angle difference $\theta_{i,k}$ is below threshold φ_a . Otherwise, the current frame is designated as a new key frame. It is formulated as follows:

$$\vartheta_i = \begin{cases} 0, & \xi_{i,k} < T, \omega_{i,k} < \varphi_d, \theta_{i,k} < \varphi_a \\ 1, & \text{otherwise} \end{cases} \quad (7)$$

where $\vartheta_i = 1$ indicates that the current frame is selected as a key frame. Then, if $\vartheta_i = 1$, we render the current frame as the key frame using the pre-trained 3DGS model $\mathcal{G}$ to obtain its RGB image I_k and depth map D_k . If $\vartheta_i = 0$, we first lift the RGB pixels of I_k to the 3D world coordinate system following eq. (3) and then project them onto the current frame following eq. (4). Next, we inpaint the blank pixels in the current projected image I_i using either of the inpainting methods to yield the completed image I'_i . Finally, these blank pixels in I_i are replaced with the corresponding pixels from I'_i . In summary, the pseudocode of the inference process is shown in algorithm 1.

IV. EXPERIMENTS

A. Experimental Setup

Datasets. We evaluate GSFlow on MipNeRF-360 [45], Tank&Temples [46], and a Custom Video Dataset. Since MipNeRF-360 does not provide continuous camera trajectories, we capture a continuous path for each scene using an interactive viewer implemented by Vulkan compute shaders. As ground truth is unavailable for these sampled cameras, we use images rendered by baseline methods as pseudo-GT for metric computation. For Tank&Temples, we extract 1,000 frames at 1920×1080 resolution per scene and compute camera parameters via COLMAP. Besides, we introduce a Custom Video Dataset captured by mobile devices, containing challenging targets like hair and leaf edges at 30 FPS. Each sequence contains nearly 1,000 images at 1280×720 resolution, with camera parameters also extracted by COLMAP. To evaluate the performance of novel view synthesis, we select every 4th image in the sequence as the test frames and use the remaining for training.

Implementation Details. Building upon the proposed geometry flow, we introduce two alternative inpainting methods: a lightweight inpainting network to achieve higher rendering quality, and a training-free filtering-based inpainting method for significantly faster acceleration. Moreover, we further accelerate the filter through a customized CUDA implementation with parallel optimization. As a plug-in, we follow the training schedule and hyperparameters of these methods. Our hyperparameters is set as: the frame projection interval $T = 3$, the camera distance threshold $\varphi_d = 0.01$ and the viewing angle threshold $\varphi_a = 1.5^\circ$. The experiments in Table IV are conducted on an RTX 2060 GPU using Vulkan compute shaders, representing computationally constrained scenarios. All other experiments are tested on a single RTX 4090 GPU under CUDA environment.

B. Comparisons with State-of-the-art

Performance under CUDA Environment. For a comprehensive evaluation, we compare our method with the state-of-the-art 3DGS rendering acceleration methods, including

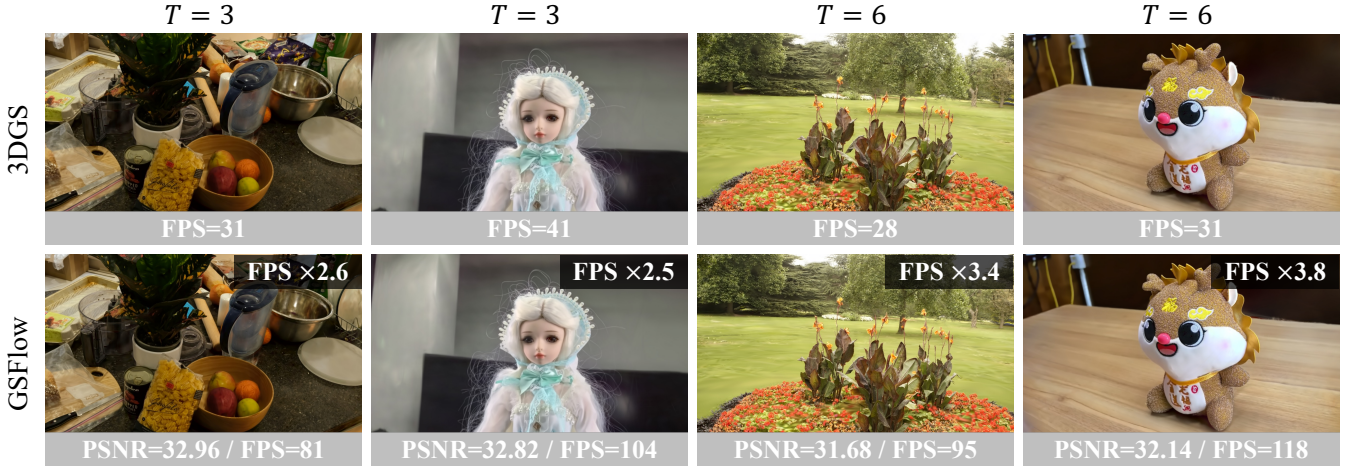


Fig. 4. **Rendering Comparisons under Vulkan Implementation.** Our method renders images with high fidelity comparable to 3DGS rasterization, even for challenging details like hair and leaves. The acceleration ratio scales linearly with the projection interval T , achieving average speedups of $\times 2.5$ at $T = 3$ and $\times 3.6$ at $T = 6$. We use images rendered by 3DGS as pseudo-GT.

TABLE II

PLUG-IN PERFORMANCE COMPARISONS. FRAME PROJECTION INTERVAL $T = 3$. THE FILTERING-BASED INPAINTING METHOD IS UTILIZED. FPS IS TESTED ON A SINGLE RTX 4090 GPU.

Methods	Custom Video Dataset			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$
3DGS [1]	33.77	0.946	0.160	169
3DGS w/ GSFlow	33.15	0.928	0.177	426
LocoGS [42]	32.62	0.933	0.191	421
LocoGS w/ GSFlow	31.87	0.919	0.211	637

TABLE III

COMPARISONS WITH OTHER PLUG-IN METHOD ON DL3DV-10K [47]. FOUR SCENES ARE RANDOMLY SELECTED FOR EVALUATION. BASE METHOD IS TC-GS. FPS IS TESTED ON A SINGLE RTX 4090 GPU.

Method	DL3DV-10K			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$
Speedy-Splat [21]	31.93	0.944	0.106	246
+ TC-GS [48]	31.92	0.944	0.105	465
+ TC-GS + Ours	31.11	0.939	0.112	752

3DGS [1], CompGS [27], Compact-3DGS [18], LightGaussian [19], EAGLES [28], Reduced-3DGS [43], PUP 3D-GS [29], Taming-3DGS [44], and LocoGS [42]. The results are reported in Table I. Notably, our approach is orthogonal to existing frame-level acceleration methods and can be applied as a plug-in to further enhance their performance. As shown in Table II, with a projection interval of $T = 3$, our method achieves a $\times 2.5$ speedup over 3DGS and a $\times 1.5$ speedup over the state-of-the-art method LocoGS. This speedup ratio will further increase linearly with the projection interval T . To further demonstrate the generalizability, we conduct experiments on the open-source video dataset DL3DV-10K [47] as shown in Table III. Compared to the existing plug-in method TC-GS [48], our approach achieves a significant improvement in inference speed with only a marginal drop in PSNR.

Performance under Vulkan Implementation. To validate our method's performance in practical applications, we deploy

TABLE IV

PERFORMANCE COMPARISONS ON MIPNERF-360 UNDER VULKAN IMPLEMENTATION. WE CAPTURE A CONTINUOUS PATH BY A INTERACTIVE VIEWER, USING IMAGES RENDERED BY BASELINES AS PSEUDO-GT. FRAME PROJECTION INTERVAL $T = 3$. THE FILTERING-BASED INPAINTING METHOD IS UTILIZED. FPS IS TESTED ON AN RTX 2060 GPU, REPRESENTING COMPUTATIONALLY CONSTRAINED SCENARIOS.

Methods	MipNeRF-360				
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$	GPU-Util $\downarrow$
3DGS [1]	-	-	-	29	93%
w/ GSFlow	32.92	0.733	0.151	79	74%
PUP 3D-GS [29]	-	-	-	72	71%
w/ GSFlow	32.32	0.697	0.184	175	44%

it on a low-power RTX 2060 GPU using Vulkan compute shaders. In this scenario, we adopt the filtering-based inpainting method, as smaller inter-frame displacements in real-world settings allow filter to achieve satisfactory inpainting. As shown in Table IV, our GSFlow achieves a $\times 2.7$ speedup over 3DGS and a $\times 2.4$ speedup over the more efficient PUP 3D-GS, while maintaining PSNR above 32dB. Furthermore, our method effectively reduces the computational load, leading to lower device power consumption. Under the same experimental setup, GPU utilization drops from 93% to 74% compared to 3DGS, and from 71% to 44% compared to PUP 3D-GS, demonstrating notable gains in energy efficiency.

Visualization Results. Visual comparisons are shown in fig. 4. Under Vulkan-based interactive scenarios with lower inter-frame disparity, our method achieves rendering quality almost consistent to 3DGS rasterization, even for challenging details like hair and leaves. Additionally, as the projection interval T increases, the speedup ratio improves linearly, reaching $\times 2.5$ at $T = 3$ and $\times 3.6$ at $T = 6$.

C. Ablation Study

Effect of Geometry Flow. As shown in the first two rows of Table V, our proposed sequence-level geometry flow yields substantial speed improvements, achieving a $\times 2.4$ acceleration on Tanks&Temples and $\times 2.5$ on the Custom Video Dataset.

TABLE V
ABLATION STUDY EVALUATED ON THE TANKS&TEMPLES AND OUR CUSTOM VIDEO DATASETS. BASELINE METHOD IS 3DGS [1]. FRAME PROJECTION INTERVAL $T = 3$. FPS IS TESTED ON A SINGLE RTX 4090 GPU.

Flow	Network	Filter	Calibration	Tanks&Temples				Custom Video Dataset			
				FPS $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
$\times$	$\times$	$\times$	$\times$	92	31.44	0.926	0.143	169	33.77	0.946	0.160
$\checkmark$	$\times$	$\times$	$\times$	249	26.42	0.839	0.246	441	27.66	0.854	0.285
$\checkmark$	$\checkmark$	$\times$	$\times$	169	31.21	0.917	0.151	257	33.54	0.930	0.172
$\checkmark$	$\times$	$\checkmark$	$\times$	224	30.53	0.906	0.159	423	32.94	0.914	0.195
$\checkmark$	$\checkmark$	$\times$	$\checkmark$	170	31.35	0.921	0.146	258	33.68	0.934	0.169
$\checkmark$	$\times$	$\checkmark$	$\checkmark$	224	30.77	0.913	0.155	426	33.15	0.928	0.177

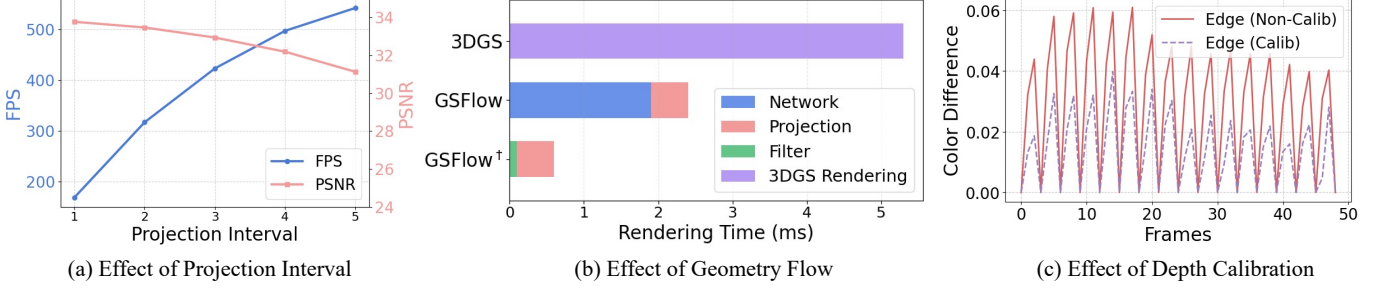


Fig. 5. (a) **Effect of Projection Interval.** The computational cost of our method is highly predictable, scaling linearly with the projection interval T , where $T = 1$ corresponds to the original 3DGS. In this work, we set $T = 3$, achieving a balance between rendering quality and acceleration performance. (b) **Effect of Geometry Flow.** The network-based variant, denoted as GSFlow, achieves a $\times 2.21$ speedup over 3DGS rasterization, while the filter-based variant, denoted as GSFlow † , achieves a remarkable $\times 8.83$ speedup. (c) **Effect of Depth Calibration.** Our self-supervised depth calibration significantly reduces color discrepancies in edge regions relative to 3DGS-rendered images, effectively mitigating edge flickering artifacts.

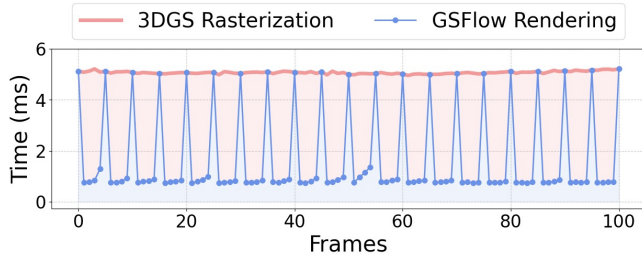


Fig. 6. **Rendering process of GSFlow.** Key frames require rendering times consistent to the baseline method while subsequent projection frames are generated at significantly higher speeds. The blue areas represent the rendering time required by our GSFlow, while the red areas indicate the accelerated portions.

fig. 6 illustrates the rendering process, where key frames require rendering times consistent to the baseline method while subsequent projection frames are generated at significantly higher speeds. Furthermore, the rendering quality and speedup ratio of our method are dependent on the projection interval T . As shown in fig. 5 (a), the computational cost scales linearly with the interval T , making it highly predictable and hardware-friendly, while the PSNR gradually decreases as T increases. To balance rendering quality and acceleration performance, we set $T = 3$.

Effect of Blank Pixel Completion. To address blank pixel completion, we propose two approaches: a trained inpainting network and a faster unsupervised filtering-based method. As shown in fig. 5 (b), the inpainting network achieves a $\times 2.21$ speedup over 3DGS rasterization, while the filter achieves a remarkable $\times 8.83$ speedup. Filter is significantly faster than the inpainting network, though the latter delivers higher

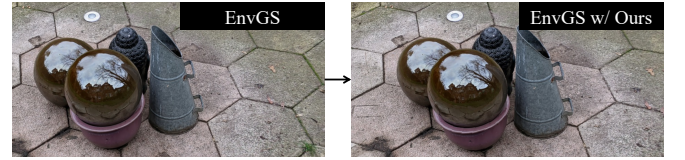


Fig. 7. **View-dependent Effects.** Specular reflections are well preserved after projection via our GSFlow.

completion quality, especially under large inter-frame displacements. As reported in Table V, the inpainting network yields a nearly 1dB PSNR improvement over the filter. Notably, in Vulkan-based deployments on low-power devices, inter-frame differences during interaction are much smaller than those in 30 FPS videos of our test dataset. As shown in Table IV, under such conditions, our filter maintains PSNR above 32dB for both 3DGS and PUP 3D-GS baselines. Besides, we evaluate our method on Ref-Real dataset following EnvGS [49], which contains abundant scenes with view-dependent effects. As shown in fig. 7, specular reflections are well preserved after projection via our GSFlow.

Effect of Self-Supervised Depth Calibration. The proposed self-supervised depth calibration enhances spatio-temporal consistency by aligning depth estimates across frames. As shown in fig. 5 (c), this approach significantly reduces color discrepancies in edge regions compared to the 3DGS-rendered images, effectively mitigating the edge flickering issue and improving the stability of GSFlow. As reported in Table V, while our depth calibration strategy yields a modest PSNR improvement of approximately +0.2dB, it leads to a substantial enhancement in visual quality during video rendering.

TABLE VI
STATISTICS OF CAMERA MOTION ON THREE DATASETS.

Datasets	Avg. Speed (m/s)	Avg. Rotation (°/s)	Failure	PSNR
Tanks&Temples	1.12	6.53°	0 / 850	30.77
Custom Dataset	1.09	18.10°	0 / 756	33.15
DL3DV-10K	1.57	21.98°	0 / 348	33.43

TABLE VII
IMPACT OF INCREASED CAMERA SPEED ON PERFORMANCE.

Avg. Speed (m/s)	Avg. Rotation (°/s)	Failure	PSNR	PSNR*	Speed-up
1.09	18.10	0 / 756	33.15	33.15	×2.52
2.09	33.70	99 / 378	30.68	32.76	×1.83
3.12	49.50	121 / 252	29.62	33.52	×1.48
4.15	65.20	127 / 189	28.76	33.58	×1.27
5.17	80.70	114 / 151	28.07	33.72	×1.19

Effects of Camera Motion Speed. We report the average camera translation speed and rotation magnitude across the three datasets in Table VI, where we define any frame with $\text{PSNR} < 28\text{dB}$ as a failure case. It can be observed that our GSFlow achieves high rendering quality at typical walking speeds. To explore the upper limit of supported camera motion, we manually accelerate camera motion on Custom Dataset videos by subsampling video frames. Our method incorporates a fall-back mechanism that, under extremely fast camera motion, can revert to the baseline model’s performance. As shown in Table VII, our method ensures rendering quality, though at the cost of reduced speedup, where PSNR* denotes accuracy when failure cases fall back to 3DGS rendering. When camera motion exceeds 5m/s, our acceleration becomes marginal ($\times 1.19$).

V. CONCLUSION

In this paper, we propose GSFlow, a novel sequence-level rendering acceleration paradigm for 3D Gaussian Splatting (3DGS), which is orthogonal to existing frame-level methods. Our GSFlow leverages inter-frame coherence to minimize the number of pixels requiring 3DGS rasterization, thereby achieving significant speedup. Specifically, GSFlow samples key frames at fixed intervals along the camera trajectory and renders their RGB and depth images via 3DGS. For subsequent frames, it lifts keyframe RGB pixels to world coordinates and projects them onto target frames, achieving geometry pixel flow. To address blank pixels from camera motion and projection errors, we propose two solutions: a lightweight inpainting network for higher rendering quality, and a training-free filtering-based method accelerated via customized CUDA parallelization for extreme speed. For edge flickering caused by depth inaccuracies, we propose a self-supervised depth calibration strategy that enhances spatio-temporal consistency by inter-frame 3D geometry alignment. Extensive experiments demonstrate the effectiveness of our proposed method.

REFERENCES

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.

[2] F. Duan, Y. Zhang, X. Li, X. Tan, J. Wang, and L. Chen, “Bugs: Universal 3d gaussian splatting with a bi-directional gaussian growing mechanism,” *IEEE Transactions on Multimedia*, 2026.

[3] Z. Huang, M. Xu, and S. Perry, “Structgs: Adaptive spherical harmonics and rendering enhancements for superior 3d gaussian splatting,” *IEEE Transactions on Multimedia*, vol. 28, pp. 1499–1510, 2026.

[4] H. Xiao, J. Zou, S. Xing, P. Li, and W. K. Kang, “Geometry-aware 3d gaussian representation for real-time rendering of large-scale scenes,” *IEEE Transactions on Multimedia*, pp. 1–18, 2026.

[5] Y. Zhao, G. Chen, B. Wu, Q. Jin, and T. Zeng, “Msa-splatting: Multi-scale adaptive gaussian splatting for high-fidelity view synthesis,” *IEEE Transactions on Multimedia*, 2026.

[6] X. Zhou, Z. Lin, X. Shan, Y. Wang, D. Sun, and M.-H. Yang, “Drivinggaussian: Composite gaussian splatting for surrounding dynamic autonomous driving scenes,” in *CVPR*, 2024, pp. 21 634–21 643.

[7] Y. Yan, H. Lin, C. Zhou, W. Wang, H. Sun, K. Zhan, X. Lang, X. Zhou, and S. Peng, “Street gaussians: Modeling dynamic urban scenes with gaussian splatting,” in *ECCV*, 2024.

[8] N. Huang, X. Wei, W. Zheng, P. An, M. Lu, W. Zhan, M. Tomizuka, K. Keutzer, and S. Zhang, “S³gaussian: Self-supervised street gaussians for autonomous driving,” 2024, arXiv:2405.20323.

[9] Z. Chen, J. Yang, J. Huang, R. de Lutio, J. M. Esturo, B. Ivanovic, O. Litany, Z. Gojcic, S. Fidler, M. Pavone *et al.*, “Omni: Omni urban scene reconstruction,” 2024, arXiv:2408.16760.

[10] Y. Jiang, C. Yu, T. Xie, X. Li, Y. Feng, H. Wang, M. Li, H. Lau, F. Gao, Y. Yang *et al.*, “Vr-gs: A physical dynamics-aware interactive gaussian splatting system in virtual reality,” in *ACM SIGGRAPH*, 2024, pp. 1–1.

[11] X. Pan, G. Huang, Z. Zhang, J. Li, H. Bao, and G. Zhang, “Robust collaborative visual-inertial slam for mobile augmented reality,” *IEEE Transactions on Visualization and Computer Graphics*, 2024.

[12] X. Huang, H. Zhu, Z. Liu, W. Lin, X. Liu, Z. He, J. Leng, M. Guo, and Y. Feng, “Seele: A unified acceleration framework for real-time gaussian splatting,” *arXiv preprint arXiv:2503.05168*, 2025.

[13] Y. Liu, C. Luo, L. Fan, N. Wang, J. Peng, and Z. Zhang, “Citygaussian: Real-time high-quality large-scale scene rendering with gaussians,” in *ECCV*, 2024, pp. 265–282.

[14] O. Shorinwa, J. Sun, and M. Schwager, “Fast-splat: Fast, ambiguity-free semantics transfer in gaussian splatting,” *arXiv preprint arXiv:2411.13753*, 2024.

[15] H. Zhao, L. Zhuang, X. Zhao, C. Zeng, H. Xu, Y. Jiang, J. Cen, K. Wang, J. Guo, S. Huang, X. Li, D. Zhao, and H. Zou, “Towards affordance-aware robotic dexterous grasping with human-like priors,” *arXiv preprint arXiv:2508.08896*, 2025.

[16] G. Chhablani, X. Ye, M. Z. Irshad, and Z. Kira, “Embodiedsplat: Personalized real-to-sim-to-real navigation with gaussian splats from a mobile device,” *arXiv preprint arXiv:2509.17430*, 2025.

[17] X. Lei, M. Wang, W. Zhou, and H. Li, “Gaussnav: Gaussian splatting for visual navigation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.

[18] J. C. Lee, D. Rho, X. Sun, J. H. Ko, and E. Park, “Compact 3d gaussian representation for radiance field,” in *CVPR*, 2024, pp. 21 719–21 728.

[19] Z. Fan, K. Wang, K. Wen, Z. Zhu, D. Xu, Z. Wang *et al.*, “Lightgaussian: Unbounded 3d gaussian compression with 15x reduction and 200+ fps,” in *NeurIPS*, vol. 37, 2024, pp. 140 138–140 158.

[20] G. Feng, S. Chen, R. Fu, Z. Liao, Y. Wang, T. Liu, B. Hu, L. Xu, Z. Pei, H. Li *et al.*, “Flashgs: Efficient 3d gaussian splatting for large-scale and high-resolution rendering,” in *CVPR*, 2025, pp. 26 652–26 662.

[21] A. Hanson, A. Tu, G. Lin, V. Singla, M. Zwicker, and T. Goldstein, “Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives,” in *CVPR*, 2025, pp. 21 537–21 546.

[22] T. Lu, M. Yu, L. Xu, Y. Xiangli, L. Wang, D. Lin, and B. Dai, “Scaffoldgs: Structured 3d gaussians for view-adaptive rendering,” in *CVPR*, 2024, pp. 20 654–20 664.

[23] Y. Chen, Q. Wu, W. Lin, M. Harandi, and J. Cai, “Hac: Hash-grid assisted context for 3d gaussian splatting compression,” in *ECCV*, 2024, pp. 422–438.

[24] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *CVPR*, 2022, pp. 10 684–10 695.

[25] Z. Yu, A. Chen, B. Huang, T. Sattler, and A. Geiger, “Mip-splatting: Alias-free 3d gaussian splatting,” in *CVPR*, 2024, pp. 19 447–19 456.

[26] M. Niemeyer, F. Manhardt, M.-J. Rakotosaona, M. Oechsle, D. Duckworth, R. Gosula, K. Tateno, J. Bates, D. Kaeser, and F. Tombari, “Radsplat: Radiance field-informed gaussian splatting for robust real-time rendering with 900+ fps,” in *3DV*, 2025, pp. 134–144.

[27] K. Navaneet, K. Pourahmadi Meibodi, S. Abbasi Koohpayegani, and H. Pirsiavash, “Compigs: Smaller and faster gaussian splatting with vector quantization,” in *ECCV*, 2024, pp. 330–349.

[28] S. Girish, K. Gupta, and A. Shrivastava, “Eagles: Efficient accelerated 3d gaussians with lightweight encodings,” in *ECCV*, 2024, pp. 54–71.

- [29] A. Hanson, A. Tu, V. Singla, M. Jayawardhana, M. Zwicker, and T. Goldstein, "Pup 3d-gs: Principled uncertainty pruning for 3d gaussian splatting," in *CVPR*, 2025, pp. 5949–5958.
- [30] H. Shuai, Y. Shi, Y. Sun, and Q. Liu, "Adversarial pruning networks for compact 3d gaussian splatting," *IEEE Transactions on Multimedia*, 2025.
- [31] X. Wang, R. Yi, and L. Ma, "Adr-gaussian: Accelerating gaussian splatting with adaptive radius," in *SIGGRAPH Asia*, 2024, pp. 1–10.
- [32] L. Radl, M. Steiner, M. Parger, A. Weinrauch, B. Kerbl, and M. Steinberger, "Stopthepop: Sorted gaussian splatting for view-consistent real-time rendering," *ACM Transactions on Graphics*, vol. 43, no. 4, pp. 1–17, 2024.
- [33] H. Gui, L. Hu, R. Chen, M. Huang, Y. Yin, J. Yang, Y. Wu, C. Liu, Z. Sun, X. Zhang *et al.*, "Balanced 3dgs: Gaussian-wise parallelism rendering with fine-grained tiling," *arXiv preprint arXiv:2412.17378*, 2024.
- [34] S. Xie, W. Zhang, C. Tang, Y. Bai, R. Lu, S. Ge, and Z. Wang, "Mesongs: Post-training compression of 3d gaussians via efficient attribute transformation," in *ECCV*, 2024, pp. 434–452.
- [35] J. C. Lee, J. H. Ko, and E. Park, "Optimized minimal 3d gaussian splatting," *arXiv preprint arXiv:2503.16924*, 2025.
- [36] S. Niedermayr, J. Stumpfegger, and R. Westermann, "Compressed 3d gaussian splatting for accelerated novel view synthesis," in *CVPR*, 2024, pp. 10 349–10 358.
- [37] Y. Chen, Q. Wu, M. Li, W. Lin, M. Harandi, and J. Cai, "Fast feedforward 3d gaussian splatting compression," *arXiv preprint arXiv:2410.08017*, 2024.
- [38] W. Morgenstern, F. Barthel, A. Hilsmann, and P. Eisert, "Compact 3d scene representation via self-organizing gaussian grids," in *ECCV*, 2024, pp. 18–34.
- [39] X. Li, G. Zheng, Y. Yu, N. Ji, and X. Li, "Relationship-incremental scene graph generation by a divide-and-conquer pipeline with feature adapter," *IEEE Transactions on Image Processing*, vol. 34, pp. 678–688, 2024.
- [40] Y. Chen, Q. Wu, W. Lin, M. Harandi, and J. Cai, "Hac++: Towards 100x compression of 3d gaussian splatting," *arXiv preprint arXiv:2501.12255*, 2025.
- [41] Y. Wang, Z. Li, L. Guo, W. Yang, A. Kot, and B. Wen, "Contextgs: Compact 3d gaussian splatting with anchor level context model," in *NeurIPS*, vol. 37, 2024, pp. 51 532–51 551.
- [42] S. Shin, J. Park, and S. Cho, "Locality-aware gaussian compression for fast and high-quality rendering," *arXiv preprint arXiv:2501.05757*, 2025.
- [43] P. Papantonakis, G. Kopanas, B. Kerbl, A. Lanvin, and G. Drettakis, "Reducing the memory footprint of 3d gaussian splatting," vol. 7, no. 1, 2024, pp. 1–17.
- [44] S. S. Mallick, R. Goel, B. Kerbl, M. Steinberger, F. V. Carrasco, and F. De La Torre, "Taming 3dgs: High-quality radiance fields with limited resources," in *SIGGRAPH Asia*, 2024, pp. 1–11.
- [45] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, "Mip-nerf 360: Unbounded anti-aliased neural radiance fields," in *CVPR*, 2022, pp. 5470–5479.
- [46] A. Knapitsch, J. Park, Q.-Y. Zhou, and V. Koltun, "Tanks and temples: Benchmarking large-scale scene reconstruction," *ACM Transactions on Graphics*, vol. 36, no. 4, pp. 1–13, 2017.
- [47] L. Ling, Y. Sheng, Z. Tu, W. Zhao, C. Xin, K. Wan, L. Yu, Q. Guo, Z. Yu, Y. Lu *et al.*, "Dl3dv-10k: A large-scale scene dataset for deep learning-based 3d vision," in *CVPR*, 2024, pp. 22 160–22 169.
- [48] T. Wang, Z. Yu, and Y. Xu, "Tc-gs: Tri-plane based compression for 3d gaussian splatting," in *ICME*, 2025, pp. 1–6.
- [49] T. Xie, X. Chen, Z. Xu, Y. Xie, Y. Jin, Y. Shen, S. Peng, H. Bao, and X. Zhou, "Envgs: Modeling view-dependent appearance with environment gaussian," in *CVPR*, 2025, pp. 5742–5751.